Summary

For quick reference, the following table offers a thorough analysis of the distinctive characteristics, applicability, examples, benefits, and drawbacks of each behavioral design pattern.

Pattern Name	Distinctive Feature	Applicability	An Example	Pros	Cons
Iterator	Provides a way to access elements of a collection sequentially without exposing its underlying representation	When you need to provide a standard way to traverse through a collection without exposing its implementation	Browsing through a playlist	Simplifies client code and hides complexity of the collection	Iterator state management can be complex
Strategy	Enables an algorithm's behavior to be selected at runtime	When there are multiple ways to achieve a task and you want to determine the algorithm at runtime	Different compression algorithms	Flexibility to switch algorithms; isolation of algorithm implementation	Complexity due to additional classes; clients must be aware of different strategies
Template Method	Defines the skeleton of an algorithm in an operation, deferring some steps to subclasses	When the core structure of an algorithm should not change, yet some steps are open to being overridden	Frameworks defining application structure	Promotes code reuse and enforces a predefined algorithm structure	Can lead to less flexibility in subclasses; harder to maintain
Observer	Notifies multiple objects about state changes	When changes in one object's state require other objects to be notified	Real-time data feeds	Loose coupling between subject and observers	Risk of unintended performance issues
Command	Encapsulates a request as an object	For operations that need to be executed, undone, or logged	User actions in GUI applications	Decouples command execution from usage	Can lead to numerous concrete command classes
State	Allows objects to alter their	For objects whose behavior	Different modes in a	Localizes state-specific	Increases the number

Pattern Name	Distinctive Feature	Applicability	An Example	Pros	Cons
	behavior when their internal state changes	depends on their state and can change at runtime	game	behaviors	of classes
Chain of Responsibility	Delegates commands to a chain of processing objects	For scenarios where multiple objects can handle a request, but the handler is unknown	Help systems in applications	Decouples request sender and receiver	Handling can become inefficient
Mediator	Centralizes complex communications between related objects	To reduce direct communications between multiple objects	Chat application	Simplifies interactions between groups of objects	Mediator can become overly complex
Memento	Captures and externalizes an object's internal state	For undo functionality or state save-and-restore	Undo feature in editors	Preserves encapsulation boundaries	Can be memory-intensive
Visitor	Separates an algorithm from the objects on which it operates	When operations need to be performed on a group of similar kinds of objects	Processing different types of document elements in a word processor	Simplifies adding new operations; centralizes related operations	Can lead to a bloated visitor interface; harder to maintain when the object structure changes

← Previous

Visitor Pattern

Mark as Completed